



CodeGuardian AI for Laravel

Analyze · Improve · Validate · Report

A production-grade, embedded static-analysis, security-audit, automated-refactoring and reporting platform that installs into any Laravel project as a Composer package — with an optional AI engine and a full web dashboard.

PHP ^8.1

Laravel 9 / 10 / 11

Flutter / Dart

20 Artisan Commands

Zero-cost Static Engine

OpenAI · Claude · Gemini

CI / SARIF / JUnit

Web Dashboard

Package

codeguardian/laravel

Document

Complete Reference & User Guide

License

MIT

Version

dev-main

Table of Contents

1	Introduction & Overview	12	Web Dashboard
2	Feature Summary	13	Foolproof Refactoring
3	Requirements	14	Filtering & Suppression
4	Installation	15	Rule Configuration
5	Quick Start	16	Continuous Integration
6	Configuration Reference	17	AI Mode
7	Command Reference (all 20)	18	Architecture & How It Works
8	What It Detects	19	Extending the Engine
9	Understanding the Results	20	Troubleshooting
10	Premium CLI Experience	21	FAQ
11	Enterprise Reporting	22	Appendix: Env, Exit Codes, Glossary

About this document

This is the complete, self-contained reference for CodeGuardian AI for Laravel. It covers installation, every configuration key, all 20 artisan commands with their options and examples, the detection catalogue, reporting formats, the web dashboard, the safe refactoring pipeline, CI/CD integration, the AI engine, internal architecture, and troubleshooting. It is intended both as an onboarding guide for new users and as a submission-ready specification.

Introduction & Overview

CodeGuardian AI is a static code analysis, security auditing, automated refactoring, and reporting platform for Laravel (PHP) and Flutter (Dart) projects. It installs as a Composer package and runs entirely through artisan commands or a built-in web dashboard.

No API key needed. No internet required. No cost. 100% embedded. The default `static` engine performs all core analysis offline and free. AI mode is an optional upgrade for richer, intent-aware explanations.

What problem does it solve?

Most teams discover architectural debt, security holes, and performance traps too late — in production, in incidents, or in painful code reviews. CodeGuardian shifts that discovery left: it scans the codebase deterministically, scores its health across six dimensions, explains every finding with a concrete fix, and can *safely* apply many fixes automatically behind a test-verified safety net.

The core workflow

1. **Analyze** — scan code and produce findings, scores, and a risk rating.
2. **Generate tests** — write PHPUnit stubs *before* any code changes.
3. **Refactor** — apply deterministic + AI-driven improvements, one file at a time.
4. **Validate** — re-run tests; auto-rollback any change that introduces a regression.
5. **Report** — emit enterprise-grade reports (console, HTML, JSON, Markdown, SARIF, JUnit).

Who is it for?

Engineering teams

Continuous code-health gating in CI, PR review automation, and onboarding-quality reports.

Tech leads & architects

Six-dimension scorecards, module dependency graphs, and trend tracking over time.

Security engineers

OWASP/CWE-mapped findings, dependency CVE audits, and SARIF for code-scanning platforms.

Individual developers

A fast inner-loop watcher, deterministic auto-fixes, and a friendly local dashboard.

Feature Summary

Capability	Description
Static analysis engine	Rule-based architecture, security, performance, tech-debt and database analyzers — offline, free, deterministic.
Security taxonomy	Every security finding mapped to OWASP Top 10 (2021), a CWE id, an indicative CVSS band, confidence, impact, effort, and breaking-change risk.
Six-dimension scorecard	Architecture, Security, Performance, Maintainability, Testability, Reliability — each 0–100 with a grade and reasoning.
Explainable risk score	0–100 urgency rating weighted by severity × confidence, with plain-English justification.
Foolproof refactoring	Test → refactor → verify loop with automatic rollback on regression; syntax-validated writes.
Deterministic auto-fixes	No-AI mechanical fixes (remove <code>dd()</code> , <code>\$request->all()</code> → <code>validated()</code> , add return types, ...).
AI engine (optional)	OpenAI / Claude / Gemini for natural-language, intent-aware review and deep refactors. Diff-only review mode for cheap PR review.
Premium CLI	Live multi-stage pipeline with spinners, ETA, per-file ticks, and an execution-stats card; TTY-aware plain mode for CI.
Reporting formats	Console, HTML (interactive), JSON, Markdown, SARIF 2.1.0, JUnit XML.
Web dashboard	Run everything from the browser, watch live progress, browse history, an Insights trends page, and an interactive findings explorer.
CI/CD native	SARIF upload, baseline/diff (fail only on new findings), JUnit panels, inline PR annotations, PR/MR comments, quality gates, and a doctor preflight.
Performance & caching	Parallel analyzers (pcntl), incremental git-scoped scans, AI response cache, and content-addressed static result cache.
Extensibility	Config-driven rule enable/disable/severity, tuning presets, custom regex rules, and external tool ingestion (PHPStan/Psalm/Clover).
Flutter / Dart	Dedicated Dart analyzer auto-enables when <code>*.dart</code> files are present.

Requirements

Requirement	Version / Notes
PHP	<code>^8.1</code>
Laravel	<code>^9.0</code> / <code>^10.0</code> / <code>^11.0</code>
API key	Not required for static mode (default). Only needed for AI / hybrid mode.
PHP extensions	<code>tokenizer</code> (required); <code>pcntl</code> (optional, for parallel analysis)
Composer dependencies	<code>illuminate/console</code> , <code>illuminate/support</code> , <code>illuminate/filesystem</code> , <code>guzzlehttp/guzzle ^7.2</code> , <code>nikic/php-parser ^4.18 ^5.0</code>
Dev dependencies	<code>orchestra/testbench</code> , <code>phpunit/phpunit</code>

The package auto-registers via Laravel package discovery (`CodeGuardian\Laravel\CodeGuardianServiceProvider`) and ships a `CodeGuardian` facade alias.

Installation

Option A — Install from GitHub (any machine)

Step 1. Add the repository to your project's `composer.json`:

```
{
  "repositories": [
    {
      "type": "vcs",
      "url": "https://github.com/bilalmahoon/codeguardian-laravel"
    }
  ]
}
```

Step 2. Require the package:

```
composer require codeguardian/laravel:dev-main
```

Step 3. (Optional) Publish the config:

```
php artisan vendor:publish --tag=codeguardian-config
```

This creates `config/codeguardian.php` where you can customise skip directories, output paths, rules, gates, and more.

Option B — Local development (path repository)

If you have the package source on disk:

```
{
  "repositories": [
    {
      "type": "path",
      "url": "/absolute/path/to/packages/codeguardian-laravel",
      "options": { "symlink": true }
    }
  ]
}
```

```
composer require codeguardian/laravel:@dev
```

One-command setup

```
php artisan codeguardian:init # publish config + scaffold CI + (optional)
git hook
php artisan codeguardian:init --ci=gitlab --hook
```

`codeguardian:init` publishes the config, writes a ready-to-use CI workflow (GitHub Actions or GitLab CI, auto-detected), and can install a pre-commit hook that runs a fast incremental scan. Nothing is overwritten without `--force`.

Verify the setup

```
php artisan codeguardian:doctor # health check with fix suggestions
```

Quick Start

The only required `.env` setting is — nothing. By default the tool runs in **static mode**, fully offline, with no API key.

```
# 1. Analyze the entire project
php artisan codeguardian:analyze

# 2. Analyze a specific area
php artisan codeguardian:analyze --path=app/Http/Controllers
php artisan codeguardian:analyze --module=User
php artisan codeguardian:analyze --api=orders

# 3. Analyze, then automatically FIX issues (safe, test-verified, auto-rollback)
php artisan codeguardian:analyze --fix

# 4. Security or performance only
php artisan codeguardian:security --fail-on=high
php artisan codeguardian:performance

# 5. Full interactive refactoring workflow
php artisan codeguardian:refactor

# 6. Open the dashboard
# https://your-app.test/codeguardian
```

When `analyze` finds issues in an interactive terminal, it offers to fix them right there (skipped automatically in CI / `--no-interaction`).

Configuration Reference

All settings live in `config/codeguardian.php`. Every key below maps to an environment variable where shown, so you can configure per-environment without editing the file.

Engine & provider

Key	Env	Default	Purpose
<code>mode</code>	<code>CODEGUARDIAN_MODE</code>	<code>static</code>	Engine: <code>static</code> (offline rules), <code>ai</code> (provider), or <code>hybrid</code> (static + AI enrichment).
<code>provider</code>	<code>CODEGUARDIAN_PROVIDER</code>	<code>openai</code>	AI provider when mode is ai/hybrid: <code>openai</code> , <code>claude</code> , <code>gemini</code> .

AI provider blocks (`openai` / `claude` / `gemini`)

Key	Env	Default
<code>key</code>	<code>CODEGUARDIAN_OPENAI_KEY</code> / <code>CODEGUARDIAN_CLAUDE_KEY</code> / <code>CODEGUARDIAN_GEMINI_KEY</code>	(falls back to <code>OPENAI_API_KEY</code> / <code>ANTHROPIC_API_KEY</code> / <code>GEMINI_API_KEY</code>)
<code>model</code>	<code>CODEGUARDIAN_OPENAI_MODEL</code> / <code>..._CLAUDE_MODEL</code> / <code>..._GEMINI_MODEL</code>	<code>gpt-4o</code> / <code>claude-opus-4-5</code> / <code>gemini-1.5-flash</code>
<code>max_tokens</code>	<code>CODEGUARDIAN_MAX_TOKENS</code>	<code>8192</code>
<code>refactor_max_tokens</code>	<code>CODEGUARDIAN_REFACTOR_MAX_TOKENS</code>	<code>16000</code>
<code>temperature</code>	—	<code>0.1</code>

Analysis

Key	Default	Purpose
<code>analysis.max_file_size</code>	<code>100000</code>	Max file size (bytes) included in AI context.
<code>analysis.parallel</code>	<code>false</code> (<code>CODEGUARDIAN_PARALLEL</code>)	Run analyzers in parallel OS processes (needs <code>pcntl</code>).
<code>analysis.max_files_per_scan</code>	<code>2000</code>	Warn/confirm above this file count (0 disables).
<code>analysis.skip_dirs</code>	<code>vendor, node_modules, .git, storage, bootstrap/cache, .dart_tool, build, .pub-cache</code>	Directories excluded from scanning.
<code>analysis.auto_validate_tests</code>	<code>false</code>	Run tests immediately after generation.
<code>analysis.max_context_tokens</code>	<code>60000</code>	Token budget per AI request.
<code>analysis.test_timeout</code>	<code>120</code>	Seconds before a test run times out.

Rules, presets, custom rules & suppression

Key	Purpose
<code>modules.paths</code>	Extra module root directories if not auto-detected (<code>Modules/</code> , <code>app/Modules/</code> , <code>app/Domain/</code>).
<code>rules</code>	Per-rule control keyed by category: <code>false</code> (disable), a severity string (override), or <code>['enabled'=>bool, 'severity'=>'..']</code> .
<code>preset</code> (<code>CODEGUARDIAN_PRESET</code>)	Overall strictness: <code>strict</code> , <code>balanced</code> (default), <code>lenient</code> . <code>rules</code> always wins.
<code>custom_rules</code>	Project-specific regex rules (<code>id</code> , <code>title</code> , <code>pattern</code> , <code>severity</code> , <code>message</code> , <code>fix</code> , <code>paths</code> , <code>exclude</code>).
<code>ignore.categories</code>	Finding categories to always suppress.
<code>ignore.paths</code>	Path substrings/globs to suppress.
<code>ignore.inline_marker</code>	Inline suppression marker (default <code>codeguardian-ignore</code>).

Caching, gates, notifications, output

Key	Default	Purpose
cache.ai_enabled	true (CODEGUARDIAN_CACHE_AI)	Cache AI responses on disk (keyed by provider+model+prompt).
cache.ttl	0 (never)	Seconds an AI cache entry stays valid.
cache.dir	storage/codeguardian/cache/ai	AI cache directory.
cache.static_enabled	false (CODEGUARDIAN_CACHE_STATIC)	Content-addressed static result cache.
cache.static_dir	storage/codeguardian/cache/static	Static cache directory.
gates	empty	CI budgets: max_critical , max_high , max_total , max_risk , min_score , min_quality .
notifications.webhook	" (CODEGUARDIAN_WEBHOOK_URL)	Default webhook endpoint for codeguardian:notify .
output.report_dir	codeguardian/reports	Where reports are saved (under storage_path()).
output.tests_dir	tests/CodeGuardian	Where generated tests are saved.
output.format	both	Default format: json html md sarif junit both all .
output.history_file	storage/codeguardian/history.jsonl	Append-only trend history.

Dashboard & refactoring

Key	Default	Purpose
<code>dashboard.enabled</code>	<code>true</code> (<code>CODEGUARDIAN_DASHBOARD</code>)	Master switch; off removes the routes.
<code>dashboard.path</code>	<code>codeguardian</code> (<code>CODEGUARDIAN_DASHBOARD_PATH</code>)	URL prefix the dashboard mounts at.
<code>dashboard.middleware</code>	<code>['web']</code>	Middleware for dashboard routes.
<code>dashboard.restrict_to_local</code>	<code>true</code> (<code>CODEGUARDIAN_DASHBOARD_LOCAL_ONLY</code>)	Local-only unless a <code>viewCodeGuardian</code> gate is defined.
<code>dashboard.runs_dir</code>	<code>codeguardian/runs</code>	Where run history + live logs are stored.
<code>dashboard.php_binary</code>	<code>auto</code> (<code>CODEGUARDIAN_PHP_BINARY</code>)	PHP binary used to launch background runs.
<code>refactor.safe_mode</code>	<code>true</code> (<code>CODEGUARDIAN_SAFE_MODE</code>)	Verify each file by tests after refactor.
<code>refactor.auto_rollback_on_fail</code>	<code>true</code> (<code>CODEGUARDIAN_AUTO_ROLLBACK</code>)	Roll back a file if it introduces a new test failure.

Command Reference

CodeGuardian registers **20 artisan commands**. All are namespaced under `codeguardian:`. Run any command with `--help` to see its full signature.

Command	What it does
<code>codeguardian:analyze</code>	Full analysis — Principal SE + Senior DevOps + Senior QA level review.
<code>codeguardian:security</code>	Security audit — SQL injection, XSS, IDOR, broken auth, secrets.
<code>codeguardian:performance</code>	Performance review — N+1, missing indexes, cache, memory.
<code>codeguardian:refactor</code>	Analyze → write tests → refactor → verify → report (full workflow).
<code>codeguardian:fix</code>	Apply safe, deterministic auto-fixes (no AI).
<code>codeguardian:test</code>	Generate PHPUnit test stubs / AI tests with assertions.
<code>codeguardian:report</code>	Generate / regenerate HTML/JSON report from a previous scan.
<code>codeguardian:audit</code>	Audit Composer dependencies for CVEs & abandoned packages.
<code>codeguardian:review</code>	AI review of changed lines only (diff-aware, low cost).
<code>codeguardian:graph</code>	Map module dependencies and detect cycles.
<code>codeguardian:watch</code>	Re-analyze changed files on every save (inner loop).
<code>codeguardian:trend</code>	Code-health trends across past runs.
<code>codeguardian:rules</code>	List rules + effective config; per-rule docs.
<code>codeguardian:explain</code>	Explain a rule — meaning, why, and fix (optional AI).
<code>codeguardian:doctor</code>	Diagnose setup & suggest fixes (CI preflight).
<code>codeguardian:init</code>	Scaffold config, CI workflow, optional git hook.
<code>codeguardian:comment</code>	Post a summary comment on the current PR / MR.
<code>codeguardian:notify</code>	Send a summary to Slack / Teams / webhook.
<code>codeguardian:config-check</code>	Validate your <code>codeguardian.php</code> configuration.
<code>codeguardian:test-impact</code>	List tests impacted by changes (+ a PHPUnit <code>--filter</code>).

`codeguardian:analyze` — the primary command

Key options:

Option	Effect
<code>--path=</code>	Directory to analyze (default: project root).
<code>--module=</code>	Analyze a single module (e.g. <code>User</code> , <code>Order</code>).
<code>--api=</code>	Analyze APIs matching a route filter (e.g. <code>GET:/api/users</code> , <code>users</code>).
<code>--agents=</code>	Run only chosen analyzers, e.g. <code>security,performance</code> .
<code>--changed</code> / <code>--since=</code>	Incremental: only files changed vs HEAD / since a git ref.
<code>--parallel</code> / <code>--no-parallel</code>	Fan analyzers across CPU cores / force sequential.
<code>--cache</code> / <code>--no-cache</code>	Reuse static results on unchanged trees / force a fresh scan.
<code>--format=</code>	<code>json html md both sarif junit all</code> .
<code>--fix</code> / <code>--refactor</code>	Auto-fix (safe) / start interactive refactoring after analysis.
<code>--fail-on=</code>	Exit non-zero at/above a severity (<code>critical high medium low</code>).
<code>--write-baseline</code> / <code>--against-baseline</code> / <code>--new-only</code>	Baseline/diff workflow (see CI section).
<code>--severity</code> , <code>--min-severity</code> , <code>--category</code> , <code>--confidence</code> , <code>--owasp</code> , <code>--cwe</code>	Combinable finding filters (see Section 14).
<code>--preset=</code>	<code>strict balanced lenient</code> tuning profile for this run.
<code>--import=</code> / <code>--coverage=</code>	Fold in PHPStan/Psalm JSON / flag untested complex code from Clover.
<code>--no-report</code> , <code>--no-history</code> , <code>--no-suppress</code> , <code>--plain</code> , <code>--annotate</code> , <code>--gate</code>	Output / behaviour toggles.

Other command signatures (selected options)

Command	Notable options
<code>codeguardian:refactor</code>	<code>--module=</code> <code>--api=</code> <code>--file=</code> <code>--files=</code> <code>--mode=auto</code> <code>--safe</code> <code>--no-backup</code> <code>--skip-tests</code> <code>--with-existing-tests</code>
<code>codeguardian:security</code>	<code>--path=</code> <code>--type=</code> <code>--output=</code> <code>--fail-on=</code> <code>--owasp=</code> <code>--cwe=</code> filters
<code>codeguardian:performance</code>	<code>--path=</code> <code>--type=</code> <code>--output=</code> <code>--category=</code> filters
<code>codeguardian:fix</code>	<code>--path=</code> <code>--module=</code> <code>--file=</code> <code>--dry-run</code>
<code>codeguardian:test</code>	<code>--path=</code> <code>--type=</code> <code>--output=</code> <code>--file=</code> <code>--dry-run</code>
<code>codeguardian:report</code>	<code>--last</code> <code>--file=</code> <code>--format=html json both</code>
<code>codeguardian:audit</code>	<code>--path=</code> <code>--format=json html md sarif</code> <code>--output=</code> <code>--fail-on=</code>
<code>codeguardian:review</code>	<code>--path=</code> <code>--since=</code> <code>--output=</code>
<code>codeguardian:graph</code>	<code>--path=</code> <code>--format=text mermaid dot</code> <code>--output=</code> <code>--fail-on-cycles</code>
<code>codeguardian:watch</code>	<code>--path=</code> <code>--interval=2</code> <code>--ext=php</code> <code>--agents=</code>
<code>codeguardian:trend</code>	<code>--limit=20</code> <code>--json</code>
<code>codeguardian:rules</code>	<code>{rule?}</code> <code>--group=</code> <code>--enabled-only</code> <code>--json</code>
<code>codeguardian:explain</code>	<code>{rule}</code> <code>--ai</code> <code>--json</code>
<code>codeguardian:doctor</code>	<code>--path=</code> <code>--json</code>
<code>codeguardian:init</code>	<code>--dir=</code> <code>--ci=github gitlab none</code> <code>--hook</code> <code>--force</code>
<code>codeguardian:comment</code>	<code>--report=</code> <code>--platform=github gitlab</code> <code>--mr=</code> <code>--dry-run</code>
<code>codeguardian:notify</code>	<code>--report=</code> <code>--url=</code> <code>--format=slack teams generic</code> <code>--dry-run</code>
<code>codeguardian:config-check</code>	<code>--json</code>
<code>codeguardian:test-impact</code>	<code>--path=</code> <code>--since=</code> <code>--tests-dir=</code> <code>--filter</code>

What It Detects

Analyzer	What it finds
Architecture	Fat controllers (>150 lines), missing service layer, direct DB in controllers, inline validation, facade overuse, business/integration logic in models, <code>env()</code> used outside <code>config/</code> (null after <code>config:cache</code>).
Security	SQL injection, XSS <code>{!! !!}</code> , hardcoded secrets & API keys, mass assignment (<code>\$request->all()</code> and <code>\$guarded = []</code>), missing authorization, debug code, insecure uploads, command injection, code injection (<code>eval</code>), insecure deserialization, weak crypto (md5/sha1 on secrets), predictable randomness for tokens, path traversal, SSRF, open redirect, blanket CSRF exemption, hard-enabled debug, dynamic file inclusion, disabled TLS verification.
Performance	N+1 queries, query-in-loop (whereIn batching), over-fetching (<code>::all()->filter()</code>), nested loops ($O(n^2)$), <code>Model::all()</code> without pagination, <code>count()</code> on loaded collections, missing eager loading, exports without chunking.
Tech Debt	Large/God classes, high cyclomatic complexity, duplicated code blocks, TODO/FIXME debt, dead code, missing return types, magic numbers, deep nesting, long parameter lists, boolean-flag parameters, empty/swallowed catch blocks.
Database	Irreversible migrations (no <code>down()</code>), unindexed foreign keys, money as <code>float / double</code> , hard-to-evolve <code>enum</code> columns, missing unique indexes on lookup columns, fully unguarded models.
Flutter / Dart	<code>print()</code> in production, <code>setState()</code> inside <code>build()</code> , oversized <code>build()</code> methods, <code>BuildContext</code> used across an async gap.
Test Generator	PHPUnit stubs from method signatures (controller, service, model, generic).
Refactoring	Auto-fixes (<code>\$request->all()</code> → <code>validated()</code> , remove <code>dd()</code> / <code>dump()</code>); reports manual fixes for the rest.

Security taxonomy. Every security finding is mapped to its OWASP Top 10 (2021) category and a CWE id, with an indicative CVSS band, a confidence level, expected impact, estimated effort, and breaking-change risk — so findings are triage-ready.

Understanding the Results

Severity levels

Level	Meaning
CRITICAL	Must fix before production (SQL injection, hardcoded secrets, ...).
HIGH	Significant architectural or security problem.
MEDIUM	Code-quality issue that should be addressed.
LOW	Minor improvement (style, naming, ...).

Score & grade

Score	Grade	Meaning
90–100	A	Excellent
80–89	B	Good
70–79	C	Acceptable
60–69	D	Needs work
< 60	F	Needs significant refactoring

Anatomy of a finding

Every finding carries the context an engineer needs to triage and fix it:

Field	Meaning
severity	critical / high / medium / low
confidence	high / medium / low — how sure the rule is
category	machine-readable type, e.g. <code>sql_injection</code> , <code>n_plus_one</code> , <code>god_class</code>
description	why it matters, with the offending line number
root_cause	the underlying reason it happens
code_snippet	the evidence (the offending line)
recommendation	the suggested fix
code_before / code_after	a concrete better implementation
impact	the expected benefit of fixing it
effort	trivial / small / medium / large
breaking_risk	none / low / medium / high
cwe / owasp	security taxonomy
principle	the engineering principle involved (SOLID:SRP, Clean Code, ...)

Risk score (explainable)

Alongside the quality scores (how good the code is), `analyze` produces a risk score (0–100, how urgently it needs attention), weighted by severity × confidence, with reasoning:

Risk Score: 62/100 (CRITICAL)

- 2 critical finding(s) drive the risk to its ceiling – these should block release.
- Concentrated in: `sql injection` (2), `n plus one` (3), `missing types` (4).
- Weighted across 14 finding(s) by severity × confidence.

Example console output

CodeGuardian AI – Analyze. Improve. Validate. Report.

📁 Scanning: /var/www/my-project/app/Http/Controllers
🔑 Project type: laravel | Scope: Full project
🔍 Engine: ⚡ Static engine (no API key needed)

Scanning files...

✓ Found 17 files (1259 lines)

ANALYSIS SUMMARY

Overall Score: 54/100 (Grade: F)

Total Issues: 23 ● 2 ● 7 ● 9 ● 5

TOP FINDINGS:

[CRITICAL] Hardcoded secret/credential detected → PaymentController.php:45

[CRITICAL] Potential SQL Injection vulnerability → ReportController.php:112

[HIGH] Fat Controller: OrderController (312 lines) → OrderController.php

Premium CLI Experience

`analyze`, `security`, and `performance` render a live, multi-stage pipeline — spinners, per-stage timers, an overall progress bar, ETA, the file currently being analyzed, and live counters — followed by an execution-stats card.

```
CodeGuardian · Static analysis · Security Analysis ██████████ 62% elapsed 193ms
· ETA 64ms
├─ ✓ Architecture Analysis 45 findings · 56ms
├─ ✓ Security Analysis 26 findings · 88ms
├─ ⚙ Performance Analysis 12/17 PaymentService.php
└─ ○ Tech-Debt Analysis
    files 17 · rule groups 4 · suggestions 71

┌ Execution stats ────────────────────┐
│ ✓ Architecture Analysis ██████████ 56ms · 22% │
│ ✓ Security Analysis ██████████ 88ms · 35% │
└ Total 254ms · 17 files · 67/s throughput · 4 rules ┘
```

It is **TTY-aware**: in CI, pipes, or log files it degrades to clean one-line-per-stage logging automatically. Force that anywhere with `--plain`.

Execution flow

```
Project Discovery → Scanning → ┌ Architecture ┐
                               │ Security      │ (live per-file ticks)
                               │ Performance   │
                               └ Tech-Debt    ┘ → Risk Scoring → Report Generation
```

Refactoring pipeline (7 stages)

Project Discovery → Static Analysis → Test Generation → Baseline Tests → Refactoring → Final Verification → Report Generation. Every recommended change ships with a **justification card**:

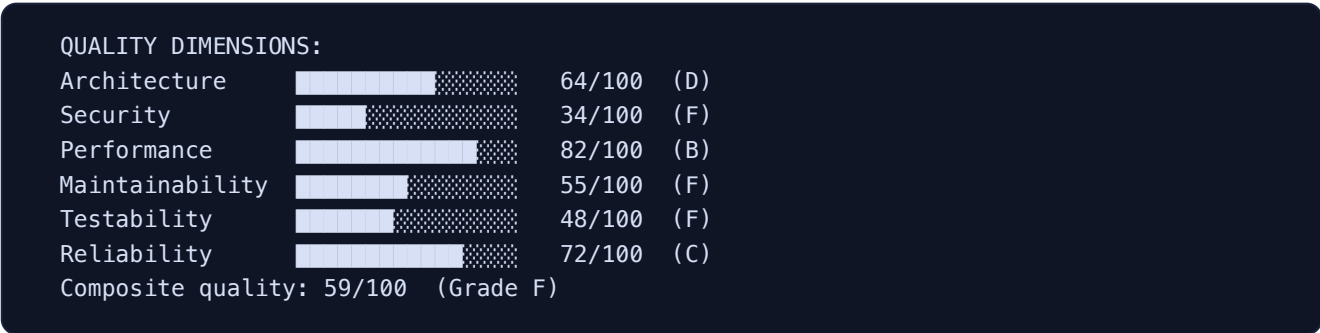
CRITICAL Possible OS command injection

Why	Untrusted data reaches an OS command sink.		
Benefit	Prevents remote code execution on the server.		
Breaking	medium	Effort medium	Confidence high
Standard	OWASP A03:2021-Injection · CWE-78		
Fix	Use <code>escapeshellarg()</code> on every argument, or <code>Symfony Process</code> .		

Enterprise Reporting

Every analysis produces an enterprise-quality report with an executive summary and a six-dimension quality scorecard, each with a 0–100 score, a letter grade, and reasoning.

Dimension	What it measures
Architecture	Structure, layering, SOLID, coupling
Security	Resistance to OWASP Top 10 / known weaknesses
Performance	Query efficiency, CPU cost, scalability
Maintainability	Complexity, duplication, readability
Testability	How easily the code can be unit-tested
Reliability	Error handling and operational fail-safes



Output formats

Format	Use case
<code>--format=json</code>	Machine-readable — full finding metadata.
<code>--format=html</code>	Rich interactive dashboard with search, severity filter, collapsible sections, and an embedded quality-score sparkline.
<code>--format=md</code>	Markdown — ideal for CI artifacts / PR comments.
<code>--format=both</code>	json + html (default).
<code>--format=sarif</code>	SARIF 2.1.0 — GitHub / GitLab / Azure code-scanning.
<code>--format=junit</code>	JUnit XML — CI "Tests" panels.
<code>--format=all</code>	json + html + md + sarif + junit.

Reports are saved under `storage/codeguardian/reports/` by default.

Web Dashboard

Prefer a UI over the terminal? CodeGuardian ships a built-in dashboard. After installing the package, open `https://your-app.test/codeguardian`.

What you can do

- **Run everything from the browser** — analyze, security audit, performance review, generate tests, and refactor. Targets are chosen from searchable dropdowns (modules, API routes, web routes, artisan commands).
- **Watch live progress** — each run executes in the background and streams its console output to the page in real time.
- **Browse full history** — every past run with its type, target, status, and saved HTML/JSON report; the history page shows a project-health snapshot (score, risk, open issues, trend).
- **Explore findings interactively** — a completed analyze run gets a Findings Explorer: filter by severity, search by title/file/category, and read each issue with its location and recommendation.
- **Track trends on the Insights page** (`/codeguardian/insights`) — quality-score and risk trend charts, severity and top-category breakdowns, quality dimensions, and hotspot files.
- **Fix from the UI** — a completed analyze run shows a "Fix these issues" panel. Select specific files or fix the whole scope; either way it launches the same safe, test-verified refactor.

Security

The dashboard is **local-only by default** (reachable only when `APP_ENV=local`). To control access:

- Define a Gate named `viewCodeGuardian` in your `AuthServiceProvider` (takes precedence; restrict by user/role), **or**
- Set `CODEGUARDIAN_DASHBOARD_LOCAL_ONLY=false` to open it (combine with the gate or your own middleware for production).

```
'dashboard' => [
    'enabled'      => env('CODEGUARDIAN_DASHBOARD', true),
    'path'         => env('CODEGUARDIAN_DASHBOARD_PATH', 'codeguardian'),
    'middleware'   => ['web'],
    'restrict_to_local' => env('CODEGUARDIAN_DASHBOARD_LOCAL_ONLY', true),
];
```

Customise the UI by publishing the views: `php artisan vendor:publish --tag=codeguardian-views`.

Foolproof Refactoring

Refactors run a **test → refactor → verify** loop so a change that breaks a test never ships:

1. Generate test stubs for the files in scope.
2. Establish a baseline (pre-existing failures are ignored — only NEW failures count).
3. Refactor each file (static auto-fixes + AI deep refactor).
4. Re-run tests; if a file's refactor introduces a new failure, it is automatically rolled back to the original.

Enabled automatically from the dashboard, with `--safe`, in `--mode=auto`, or via config (`refactor.safe_mode`, `refactor.auto_rollback_on_fail`).

The full interactive workflow

- | | |
|--------------------|--|
| 1. Analyze code | → find all issues |
| 2. Show report | → ask confirmation to proceed |
| 3. Generate tests | → write test stubs BEFORE changing any code |
| 4. Run tests | → establish baseline (passes/fails) |
| 5. Refactor | → fix one file at a time (ask per file in interactive mode) |
| 6. Run tests again | → verify nothing broke (rollback / continue / stop on failure) |
| 7. Final report | → before/after summary saved to storage/codeguardian/reports/ |

Targeting

```
php artisan codeguardian:refactor --module=Order
php artisan codeguardian:refactor --api=v1/auth/login --safe
php artisan codeguardian:refactor --file=app/Services/AuthService.php
php artisan codeguardian:refactor --
files="app/Services/AuthService.php,app/Http/Controllers/Auth/LoginController.php" --safe
```

`--files=` / `--file=` trace each file's dependency chain (via Reflection) and refactor them together.

Deterministic fixes (no AI)

```
php artisan codeguardian:fix --dry-run    # preview the safe mechanical fixes
php artisan codeguardian:fix              # apply (keeps a .cgbak backup per file)
```

`codeguardian:fix` applies only deterministic, rule-based transforms; every change is syntax-validated before writing, so the engine never produces broken PHP.

Filtering & Suppression

Filters (change what's shown)

Filters are AND-combined; each CSV value is OR-combined.

Flag	Example	Keeps
<code>--severity</code>	<code>--severity=critical,high</code>	only those severities
<code>--min-severity</code>	<code>--min-severity=high</code>	at or above that severity
<code>--category</code>	<code>--category=sql_injection,n_plus_one</code>	categories matching any substring
<code>--confidence</code>	<code>--confidence=high</code>	findings at that confidence
<code>--owasp</code>	<code>--owasp=A03,A01</code>	security findings in those OWASP categories
<code>--cwe</code>	<code>--cwe=CWE-89</code>	findings with that CWE id

Suppression (remove entirely — from scores, reports, CI exit codes)

```
// config/codeguardian.php
'ignore' => [
  'categories' => ['magic_numbers', 'commented_code'],
  'paths'      => ['database/migrations/', 'tests/*'],
],
```

Inline, right in the source:

```
$hash = md5($value);           // codeguardian-ignore weak_cryptography
// codeguardian-ignore-file    # suppresses every finding in this file
```

A bare `// codeguardian-ignore` suppresses any finding on that line; adding category names limits it. Use `--no-suppress` to temporarily see everything.

Rule Configuration

Take full control of the engine from config — no code changes. Rules are keyed by their finding category; list them all with `php artisan codeguardian:rules`.

```
// config/codeguardian.php
'rules' => [
    'magic_numbers' => false,           // turn a rule off entirely
    'missing_types' => 'low',           // downgrade severity
    'n_plus_one'    => 'critical',       // upgrade severity
    'todo_debt'     => ['enabled' => true, 'severity' => 'low'],
],
```

```
php artisan codeguardian:rules           # full catalog + effective state
php artisan codeguardian:rules --group=security # one analyzer group
php artisan codeguardian:rules sql_injection # full docs for one rule
```

Severity overrides apply *before* scoring, filters, and CI gates.

Tuning presets

```
php artisan codeguardian:analyze --preset=strict # maintainability debt first-class
php artisan codeguardian:analyze --preset=lenient # mute the noisiest low-value rules
```

Custom rules (no PHP)

Define project-specific detection rules in `config('codeguardian.custom_rules')`. Each rule scans file contents with a regex and emits a finding (severity, message, fix, path filters). They flow through scoring, filters, suppression, and CI gates like built-in rules.

Per-rule documentation

Every rule ships with a short explainer — what it means, why it matters, how to fix it, and authoritative references (OWASP / CWE / Laravel docs). HTML reports link each finding to its docs via a "Learn more" link.

Continuous Integration

SARIF — GitHub / GitLab / Azure

`--format=sarif` emits schema-valid SARIF 2.1.0 with `security-severity`, stable `partialFingerprint`s, CWE/OWASP `helpUri`s, and one-click `fixes[]` where a concrete replacement exists.

```
# GitHub Actions
- run: php artisan codeguardian:analyze --format=sarif --plain
- uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: storage/codeguardian/reports
```

Baseline / diff — fail only on new findings

```
# once, on a clean main branch — commit the file
php artisan codeguardian:analyze --write-baseline

# in CI on every PR — non-zero exit only if NEW findings appear
php artisan codeguardian:analyze --against-baseline --new-only --plain
```

Fingerprints are line-number-independent (category + file + title + normalised snippet). The summary reports new / existing / fixed counts.

Quality gates / budgets

```
'gates' => [
  'max_critical' => 0,
  'max_high'     => 5,
  'max_total'    => 200,
  'max_risk'     => 60,    // 0-100
  'min_score'    => 70,    // 0-100
  'min_quality'  => 70,    // composite 0-100
],
```

Any breach prints the violated budgets and exits non-zero.

Other CI integrations

- **Inline PR annotations** — `--annotate` (auto in GitHub Actions) renders findings on the "Files changed" view, no upload.
- **PR / MR comment** — `codeguardian:comment` posts a Markdown summary (auto-detects GitHub/GitLab; updates its previous comment).

- **JUnit panels** — `--format=junit` lights up native test-report UIs.
- **Dependency audit** — `codeguardian:audit --fail-on=high` scans `composer.lock` for CVEs and abandoned packages.
- **Doctor preflight** — `codeguardian:doctor --json` returns non-zero on hard failures.
- **Test impact** — `codeguardian:test-impact --filter` emits a PHPUnit filter to run only affected tests.

Exit codes: `0` = success (no issues at/above `--fail-on`); `1` = issues found at the specified level / gate breached / hard error.

AI Mode (Optional)

For richer, intent-aware recommendations, add an API key and switch mode:

```
CODEGUARDIAN_MODE=ai
CODEGUARDIAN_PROVIDER=openai      # or: claude, gemini
CODEGUARDIAN_OPENAI_KEY=sk-...
```

```
php artisan codeguardian:analyze --mode=ai
php artisan codeguardian:analyze --mode=hybrid # static first, AI enriches
```

Provider	Default model	Notes
OpenAI	gpt-4o	General-purpose, strong code reasoning.
Claude (Anthropic)	claude-opus-4-5	Large output budget — ideal for full-file refactors.
Gemini (Google)	gemini-1.5-flash	Free tier available; fast.

Cost controls

- **AI response cache** — identical prompts are served from disk for \$0 (model is part of the key).
- **Diff-only review** — `codeguardian:review` sends only changed lines, far cheaper than re-reviewing whole files.
- **Usage meter** — every AI run prints token usage and estimated cost (💰 AI usage: ... ~\$0.0123); tune the price table via `config('codeguardian.pricing')` .

Static mode finds the same structural issues as AI mode for most common patterns. AI mode adds natural-language summaries and catches subtler logic issues that require understanding program intent. If a key is missing or a model 404s, AI commands fall back to static.

Architecture & How It Works

The package is a rule-based static-analysis engine built on PHP token scanning, AST parsing (`nikic/php-parser`), and pattern matching — with an optional AI layer on top.

Core components

Component	Responsibility
<code>StaticOrchestrator</code>	Runs all analyzers, merges results, computes scores, drives deterministic auto-fixes, and supports parallel execution.
<code>ArchitectureAnalyzer</code>	Lines/methods per class, DB-in-controller detection, FormRequest usage, layering checks.
<code>SecurityAnalyzer</code>	Injection/secret/XSS patterns mapped to OWASP/CWE with confidence.
<code>PerformanceAnalyzer</code>	N+1, queries-in-loops, unpaginated reads, collection mis-use.
<code>TechDebtAnalyzer</code>	Cyclomatic complexity, duplication, nesting, TODOs, dead code.
<code>DatabaseAnalyzer</code>	Migration/schema/model review.
<code>DartAnalyzer</code>	Flutter/Dart-specific rules (auto-enabled with <code>*.dart</code>).
<code>StaticTestGenerator</code>	PHPUnit stubs from class/method signatures with correct namespaces.
<code>RefactorAgent</code> / <code>AIClient</code>	AI-driven deep refactoring and provider abstraction (OpenAI/Claude/Gemini) with response caching and robust JSON extraction.
<code>RouteResolver</code> / <code>DependencyTracer</code>	Laravel-router URI → controller resolution and Reflection-based call-chain tracing for <code>--api</code> / <code>--file</code> scope.
<code>QualityScorer</code> / <code>RiskScorer</code>	Six-dimension scorecard and explainable risk rating.
Supporting utilities	<code>Baseline</code> , <code>Suppressor</code> , <code>RuleRegistry</code> , <code>HistoryStore</code> , <code>DashboardInsights</code> , <code>QualityGate</code> , <code>DependencyAudit</code> , <code>DependencyGraph</code> , <code>GitDiff</code> , <code>Coverage</code> , <code>ExternalFindings</code> , <code>UsageMeter</code> , formatters (SARIF/JUnit/Markdown/HTML/annotations).

Repository layout

```
codeguardian-laravel/  
├─ src/  
│   ├── Analyzers/      static analyzers + Severity + result types  
│   ├── Agents/         AI agents (RefactorAgent, base agent)  
│   ├── Commands/       20 artisan commands  
│   ├── Console/        live pipeline / progress rendering  
│   ├── Http/           dashboard controllers + middleware  
│   ├── Support/        engine utilities, formatters, stores  
│   └─ CodeGuardianServiceProvider.php  
├─ config/codeguardian.php  
├─ resources/views/     dashboard Blade views  
├─ routes/web.php       dashboard routes  
├─ tests/               Unit + Integration (Orchestra Testbench)  
└─ composer.json
```

The test suite covers analyzers, support utilities, commands, and the dashboard (Unit + Integration), guarding against regressions and false positives.

Extending the Engine

Adding a rule is a localized change to one analyzer:

1. Open the relevant analyzer in `src/Analyzers/`.
2. Add a `private function checkYourRule(string $filePath, string $content): void` and call it from `analyze()`.
3. Emit findings via `AnalysisResult::make()` with full metadata.
4. Add a unit test in `tests/Unit/Analyzers/` — positive fixture finds it, clean fixture does not.

```
$this->addResult(AnalysisResult::make(
    category:      'my_rule',
    severity:      'high',
    title:         'Short, specific title',
    description:    'Why it matters, with the line number.',
    file:          $filePath,
    lineStart:     $lineNum + 1,
    recommendation: 'The concrete fix.',
    confidence:    'high',
    impact:        'What fixing it buys you.',
    effort:        'small',
    breakingRisk:  'low',
    cwe:           'CWE-123',           // security rules
    owasp:         'A01:2021-...',     // security rules
    principle:     'SOLID:SRP',        // design rules
));
```

Design principle for new rules: **high confidence, low false positives**. When a pattern is ambiguous, gate it on a taint/context signal and set `confidence` accordingly rather than emitting a noisy finding.

For most teams, **custom regex rules** in config (Section 15) are enough — no PHP required.

Troubleshooting

Symptom	Cause / Fix
<code>No issues found</code> on a large codebase in seconds	You're in <code>static</code> mode (expected — it's fast). For deeper, intent-aware review add an API key and use <code>--mode=hybrid</code> .
<code>--api=...</code> analyzes unrelated files	Route resolution falls back to regex if the Laravel router can't resolve the URI. Verify the route with <code>php artisan route:list</code> .
Reports not generated	Check write permissions on <code>storage/codeguardian/</code> , or pass <code>--output=</code> to a writable directory.
Too much noise	Use filters: <code>--min-severity=high</code> , <code>--confidence=high</code> , or <code>--category=</code> .
AI mode falls back to static	No/invalid API key, or the model returned 404. Check <code>CODEGUARDIAN_PROVIDER</code> and the matching key.
Dashboard returns 403	It's local-only by default. Set <code>APP_ENV=local</code> , define a <code>viewCodeGuardian</code> gate, or set <code>CODEGUARDIAN_DASHBOARD_LOCAL_ONLY=false</code> .
Live UI looks garbled in CI logs	CI isn't a TTY; the tool auto-switches to plain logging. Force with <code>--plain</code> .
<code>env()</code> finding in app code	Intentional rule — <code>env()</code> returns null after <code>config:cache</code> . Move the value into <code>config/*.php</code> and read via <code>config()</code> .

Frequently Asked Questions

Does it need an API key?

No. The default `static` engine runs fully offline with zero cost. An API key only unlocks the optional `hybrid` / `ai` modes.

Will it modify my code during analysis?

No. `analyze`, `security`, and `performance` are strictly read-only. Only `refactor` and `fix` write — and refactor runs a test-first safety net with automatic rollback.

How do I reduce noise?

Use filters (`--min-severity=high`, `--confidence=high`, `--category=`) or suppression (config + inline comments).

How do I run only part of the analysis?

`--agents=security,performance`, or the dedicated `codeguardian:security` / `codeguardian:performance` commands.

Can I add my own rules?

Yes — config-based custom regex rules (no PHP), or a small method on an analyzer plus a unit test.

Which Laravel / PHP versions are supported?

PHP `^8.1` and Laravel 9 / 10 / 11.

How do I make CI fail on regressions only?

Record a baseline once, then run `--against-baseline --new-only --fail-on=high` on PRs.

Appendix

Environment variable reference

Variable	Default
CODEGUARDIAN_MODE	static
CODEGUARDIAN_PROVIDER	openai
CODEGUARDIAN_OPENAI_KEY / _CLAUDE_KEY / _GEMINI_KEY	—
CODEGUARDIAN_OPENAI_MODEL / _CLAUDE_MODEL / _GEMINI_MODEL	gpt-4o / claude-opus-4-5 / gemini-1.5-flash
CODEGUARDIAN_MAX_TOKENS	8192
CODEGUARDIAN_REFACTOR_MAX_TOKENS	16000
CODEGUARDIAN_PARALLEL	false
CODEGUARDIAN_PRESET	balanced
CODEGUARDIAN_CACHE_AI / CODEGUARDIAN_CACHE_TTL	true / 0
CODEGUARDIAN_CACHE_STATIC	false
CODEGUARDIAN_WEBHOOK_URL	—
CODEGUARDIAN_SAFE_MODE / CODEGUARDIAN_AUTO_ROLLBACK	true / true
CODEGUARDIAN_DASHBOARD / _DASHBOARD_PATH / _DASHBOARD_LOCAL_ONLY	true / codeguardian / true
CODEGUARDIAN_PHP_BINARY	auto

Glossary

Term	Meaning
Static mode	Offline, rule-based analysis. Default. No API key.
Hybrid mode	Static analysis first, then AI enrichment of findings.
Finding	A single detected issue with full triage metadata.
Risk score	0–100 urgency rating (severity × confidence), with reasoning.
Quality dimension	One of six 0–100 scored aspects of code health.
Baseline	A snapshot of existing findings so CI fails only on new ones.
Quality gate	A configured budget that fails the build when breached.
Safe mode	Test-verified refactoring with automatic rollback.
SARIF	Static Analysis Results Interchange Format — code-scanning standard.

Key file locations

Path	Contents
<code>config/codeguardian.php</code>	All configuration.
<code>storage/codeguardian/reports/</code>	Generated reports (JSON/HTML/MD/SARIF/JUnit).
<code>storage/codeguardian/history.jsonl</code>	Trend history.
<code>storage/codeguardian/runs/</code>	Dashboard run history + live logs.
<code>storage/codeguardian/cache/</code>	AI + static result caches.
<code>tests/CodeGuardian/</code>	Generated test stubs.

License: MIT. **Package:** `codeguardian/laravel`. CodeGuardian AI — Analyze. Improve. Validate. Report.